CPU 스케줄링 복합문제 풀이

5장 CPU 스케줄링

PDF

1) 복합문제 1~2번(pptx의 41번~50번 슬라이드, 교재 291p) 문제 풀이 완벽히 이해해야 하는데 설명 좀해줘

분석

제시해주신 PDF 파일의 41번부터 50번 슬라이드에 해당하는 **복합문제 1번과 2번**의 풀이 과정을 상세히 설명해 드리겠습니다. 이 문제들은 다양한 CPU 스케줄링 알고리즘의 작동 방식을 이해하고, 성능 지표인 **평균 대기 시간**을 계산하는 것이 핵심입니다.

[복합문제 1] (41~45번 슬라이드)

이 문제는 동일한 조건에서 알고리즘에 따라 결과가 어떻게 달라지는지 비교합니다.

1. 문제 조건

- **스레드 정보:** T1(4ms), T2(3ms), T3(1ms), T4(3ms)
- **도착 시간:** T1(0), T2(1), T3(2), T4(5)
- **성능 지표:** 평균 대기 시간이 짧을수록 좋은 성능으로 평가합니다.

2. 알고리즘별 풀이

2. 알고리즘별 풀이

- **FCFS (선입선처리):** 도착한 순서대로 CPU를 할당합니다.
  - 순서: T1 → T2 → T3 → T4
  - 대기 시간: T1(0), T2(3), T3(5), T4(3)
  - **평균 대기 시간:** (0 + 3 + 5 + 3) / 4 = **2.75ms**
- **SJF (최단 작업 우선):** 준비 큐에 있는 것 중 실행 시간이 짧은 것을 먼저 선택합니다 (비선점).
  - 0ms에 T1 시작 → 4ms에 종료. 그 사이 T2, T3 도착.
  - T2(3ms)와 T3(1ms) 중 짧은 T3 먼저 실행 → T2 실행.
  - **평균 대기 시간: 2.25ms**
- **SRTF (최소 잔여 시간 우선):** 새로운 스레드 도착 시 남은 시간이 더 짧으면 CPU를 뺏습니다 (선점).
  - **평균 대기 시간: 2ms** (가장 효율적임)
- **RR (라운드 로빈):** 타임 슬라이스(Time Slice) 단위로 돌아가며 실행합니다.
  - **타임 슬라이스 2ms일 때:** 평균 대기 시간 **3ms**
  - **타임 슬라이스 1ms일 때:** 평균 대기 시간 **3.25ms** (문맥 교환 오버헤드 증가)

[복합문제 2] (46~50번 슬라이드)

이 문제는 **스케줄링 오버헤드(Scheduling Overhead)** 개념을 포함하여 실제 실행 시간을 계산하는 것이 특징입니다.

1. 주요 개념: 오버헤드 계산

- **스케줄링 오버헤드:** 스케줄링 시간 + 컨텍스트 스위칭 시간을 의미합니다. +1
- 예를 들어, RR 스케줄링에서 타임 슬라이스가 끝날 때마다 발생하는 비용을 합산해야 합니다.

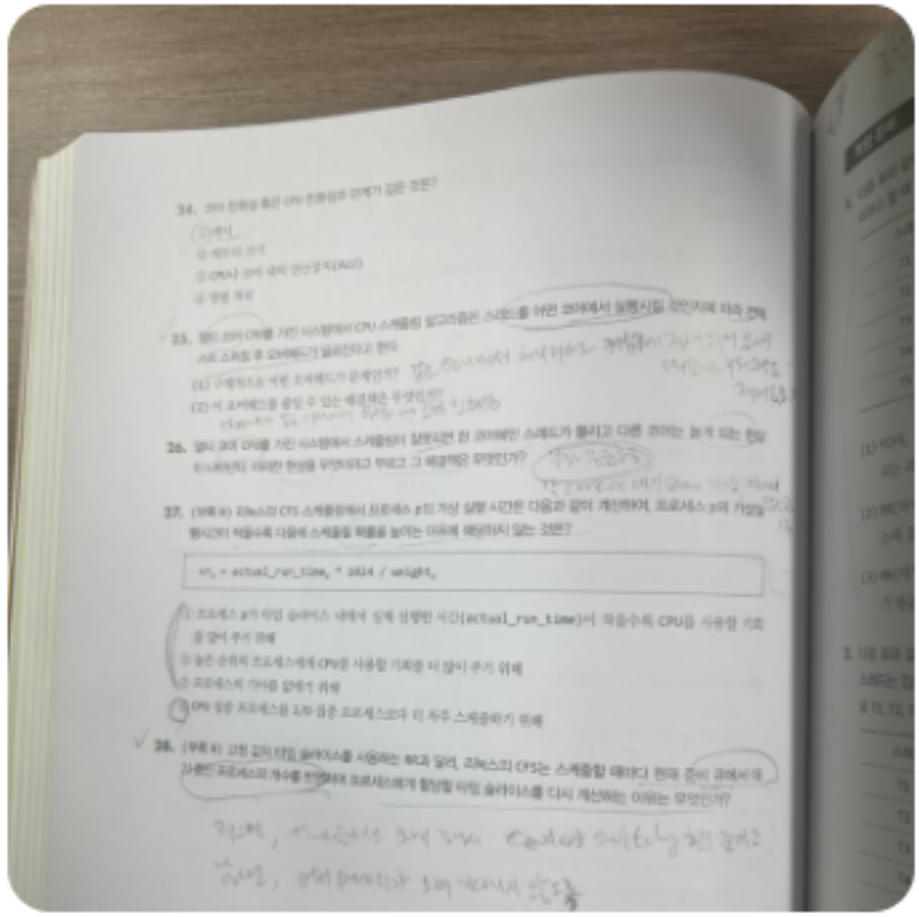
2. 풀이 포인트 (45번 슬라이드 사례)

- **조건:** T1~T5의 실행 시간 합이 18ms이고, 스케줄링이 총 9번 발생하며, 번당 오버헤드가 0.1ms인 경우.
- **계산:**
  1. 순수 실행 시간: 18ms
  2. 총 오버헤드: 9번 × 0.1ms = 0.9ms
  3. **최종 총 실행 시간:** 18 + 0.9 = **18.9ms**

3. 이해를 위한 팁

- **RR 스케줄링의 딜레마:** 타임 슬라이스가 너무 작으면 응답성은 좋아지지만, 위 계산처럼 오버헤드가 빈번하게 발생하여 전체 시스템 성능(처리율)이 떨어집니다. +1
- **평균 대기 시간의 의미:** 사용자 입장에서는 내가 얼마나 기다렸는지가 중요하므로 이 값이 작을수록 "빠르다"고 느낍니다.

이 문제들을 통해 **어떤 알고리즘이 특정 상황에서 가장 효율적인지**, 그리고 **이론과 달리 실제 시스템에서는 오버헤드라는 변수가 어떻게 작용하는지**를 이해하는 것이 학습 목표입니다.



연습문제 27번, 28번에서 제기하는 CFS(Completely Fair Scheduling)의 필요성을 설명해 보세요.

사용자께서 올려주신 교재 이미지와 연습문제 내용을 바탕으로, **CFS(Completely Fair Scheduler)**가 왜 필요한지 그 핵심 이유를 정리해 드릴게요.

연습문제 27번과 28번은 주로 기존 스케줄링 방식의 한계와 CFS가 이를 어떻게 극복하려 하는지를 묻고 있습니다.

1. 기존 스케줄러의 한계 (연습문제 27번 관련)

CFS 이전의 리눅스 스케줄러(O(1) 스케줄러 등)는 **우선순위(Priority)** 기반으로 CPU 시간을 배분했습니다. 하지만 여기에는 몇 가지 고질적인 문제가 있었죠.

- **불공평한 시간 배분:** 높은 우선순위를 가진 프로세스가 CPU를 과도하게 점유하면, 낮은 우선순위 프로세스는 실행 기회를 거의 잡지 못하는 **기아(Starvation)** 현상이 발생하기 쉬웠습니다.
- **복잡한 휴리스틱:** 인터랙티브한 작업(마우스 클릭 등)에 반응성을 높이기 위해 복잡한 계산식을 사용했는데, 이게 오히려 예측 불가능한 성능 저하를 불러오기도 했습니다.

2. CFS의 핵심 철학: "완전한 공평함" (연습문제 28번 관련)

CFS는 이름 그대로 모든 프로세스에게 **"CPU를 마치 1/N씩 공평하게 나눠 쓰는 것처럼 보이게 하겠다"**는 목표로 설계되었습니다.

- **vruntime(Virtual Runtime) 도입:** CFS의 가장 핵심적인 개념입니다. 프로세스가 실제로 CPU를 사용한 시간을 기록하되, 우선순위가 높으면 이 시간을 '천천히' 흐르게 하고, 낮으면 '빨리' 흐르게 가중치를 둡니다.
- **적응형 스케줄링:** 스케줄러는 항상 `vruntime` 이 가장 적은(즉, 가장 CPU를 못 쓴) 프로세스를 선택해 실행합니다. 이를 통해 어떤 프로세스도 소외되지 않는 공평함을 구현합니다.

---

3. 왜 CFS가 필요한가? (결론)

1. **예측 가능한 공정성:** 우선순위가 낮더라도 시간이 지나면 결국 `vruntime`이 가장 작아지는 순간이 오기 때문에, 반드시 실행 기회를 보장받습니다.
2. **효율적인 관리:** CFS는 **Red-Black Tree**라는 자료구조를 사용하여 다음에 실행할 프로세스를 찾는 시간 복잡도를 $O(\log N)$으로 유지합니다. 프로세스가 많아져도 안정적인 성능을 냅니다.
3. **인터랙티브 반응성 향상:** 잠깐 잠들었다가 깨어난(I/O 대기 등) 프로세스는 그동안 CPU를 안 썼으므로 `vruntime`이 매우 낮습니다. 따라서 깨어나자마자 즉시 CPU를 할당받을 확률이 높아져 사용자 입장에서 반응이 빠르다고 느끼게 됩니다.

   정리하자면: CFS는 특정 프로세스가 CPU를 독점하는 것을 막고, 시스템의 모든 작업이 자신의 가중치에 비례하여 **정확하고 공정하게** CPU 자원을 나눠 갖도록 하기 위해 도입된 현대적인 해결책입니다.

더 궁금한 계산 과정이나 특정 메커니즘이 있다면 말씀해 주세요!